

s1.05 — Internal metrics ingestion service

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Agent concurrency and aggregation — IC3 baseline](#)
- [Delivery, time, and replay — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: every host posts samples straight to the time-series database and retries on failure.
 - First break: a database slowdown backs up into `100,000` hosts, retries double-count counters, and an in-memory queue acknowledgment loses points on crash.
 - Second break: a consumer crashes after writing but before committing progress, rollups double-add, averaged percentiles lie, and one tenant's label explosion evicts everyone's index.
 - Minimum primitives
 - A swap-buffer agent
 - A partitioned durable log acknowledged after quorum
 - Stable point identity
 - An atomic dedup-plus-update sink boundary
 - A per-tenant cardinality budget.
- 🗣️: "Acknowledge at the durable log, not the database; then make every downstream write idempotent so replay costs nothing but time."
- Opening script
 - "I will separate durable ingestion from query freshness and show how batching, identity, and bounded recovery fit together."
 - "First I confirm metric types, the acceptable loss policy, and how stale a dashboard may be."
 - "Then I size the log, the catch-up rate, and the cardinality budget from one million points per second."
 - "The deep dive is the agent flush race and the replay-safe sink boundary."
 - "I will close with the outage drill, tenant isolation, and where I would change the design if the data became billing-grade."
- Numerical targets below are interview assumptions unless a reference explicitly supplies them.

- Validate performance targets with the actual payload and deployment topology.

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Metric semantics, loss policy, primary invariant
Estimates and contract	11:00–17:00	Points/s, backlog math, batch API semantics
Architecture	17:00–30:00	Diagram, log/consumer/query ownership, one request trace
Deep dive	30:00–45:00	Agent flush race, point identity, replay-safe sink
Operations and tradeoffs	45:00–50:00	Failure table, outage drill, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Which metric types?	Counters, gauges, histograms	Each type needs its own aggregation; one count/sum/max record is wrong
Loss policy before acceptance?	Operational telemetry; bounded counted drops allowed at the agent	Permits drop-on-full spool; billing-grade data would forbid it
Durability versus freshness priority?	Accepted data must survive sink outages; freshness may degrade	Forces a durable log before any sink
Query workload?	Dashboards over recent bounded ranges; no full-text log search	Rollups plus raw; series fan-out caps
Multi-tenant?	Yes; internal teams as tenants mapped to compartments	Tenant sits in every series key, budget, and query path
Out-of-order tolerance?	Bounded, about 5 min	Lateness window, provisional rollups, versioned repair
Who owns the agent?	The platform ships the agent on every host	Producer incarnation and batch identity are ours to define

- Functional scope
 - Accept batched points from an agent on every host.
 - Clarify the metric semantics.
 - Counters represent accumulated events and may reset on process restart.
 - Gauges represent sampled current values.
 - Histograms represent distributions through mergeable bucket counts.
 - Do not aggregate every metric into the same count/sum/max record.
 - Persist accepted points durably before any sink applies them.
 - Serve bounded dashboard queries over raw points and rollups.
 - Support authorized replay of a source range into a sink.
- Assumed scope
 - 100,000 hosts.
 - 100 active series/host.
 - One point per series every 10 s.
 - Dashboard queries over recent bounded time ranges.
 - Full-text log search is separate.

Requirement	Target	Enforced by
Sustained ingest	1,000,000 points/s	Partitioned log sized with headroom; per-tenant admission budget
Ingestion availability	99.95%	Stateless gateway fleet across fault domains; replicated log quorum
Batch acknowledgment latency	p99 < 250 ms after gateway receipt	Acknowledge after log quorum only; no synchronous database write
Accepted-data freshness	p99 < 30 s	Consumer lag alarm; catch-up capacity of 1.5× live rate
Bounded dashboard query	p99 < 2 s	Series fan-out and point caps; rollups; query budget per tenant
Replay log retention	24 h	Retention configured above outage plus catch-up plus margin
Lateness window	5 min (assumption)	Provisional rollups until the window closes
Per-tenant active-series budget	1 million (assumption)	Admission check at the gateway before a new series is created
Control-plane independence	30 min outage drill with valid admission budgets	Gateways retain policy only within its declared 30 min lease; expiration invokes the stream fail policy

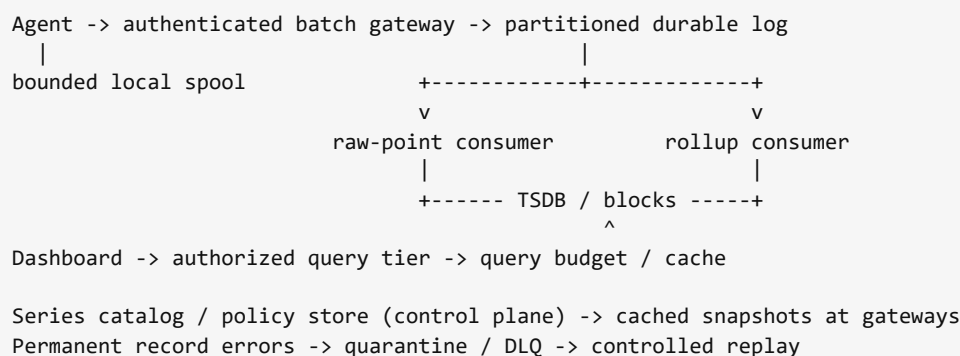
- Invariants
 - Acknowledged acceptance means the batch reached the chosen durable boundary.
 - A consumer retry must not double-add an already applied aggregate.
 - Tenant identity is part of every series and access path.
 - Queue and cardinality bounds are explicit.
 - Data before durable acceptance may be dropped only under the documented metrics policy.
- Non-goals
 - Billing or metering records with a no-loss contract.
 - Full-text log search or trace storage.
 - Cross-tenant metric federation.

Capacity and consequence — IC3 baseline

- Input
 - $100,000 \times 100 / 10 = 1,000,000$ points/s.
 - At an assumed compact 50 B/point, raw input is 50 MB/s.
 - Raw daily volume is $50 \text{ MB/s} \times 86,400 \approx 4.32 \text{ TB/day}$.
 - Three replicated copies imply approximately 12.96 TB for a day before indexes and overhead.
 - Compression is workload-dependent and must be measured.
 - Decision: the log must be partitioned and retention-bounded
 - 24 h of replicated raw log is on the order of 13 TB, so retention is a capacity line item, not a default.
- Batching
 - One flush per host every 10 s gives 10,000 batch requests/s.
 - In this sampling scenario, batching reduces requests, not the one million logical points.
 - Local pre-aggregation reduces points only when the original measurements and query semantics allow it.
 - Decision: the gateway is sized for 10,000 requests/s of about 5 KB each
 - The log and sinks are sized for one million points, and no one may quote the smaller number for the larger system.
- Recovery capacity
 - A 2 h sink outage creates 7.2 billion points of backlog.
 - A recovery rate of 1.5 million/s leaves 0.5 million/s for catch-up.
 - Catch-up takes $14,400 \text{ s} = 4 \text{ h}$ while live traffic continues.
 - The oldest affected record waits about six hours from outage start to full recovery.

- Retention must cover outage plus catch-up plus operational margin.
- Decision: provision sink write capacity at 1.5x live rate and set log retention to 24 h
 - A 2 h outage then consumes 6 h of the retention window and leaves margin.
- Series cardinality
 - 10 million active series can dominate TSDB memory and indexes.
 - Bound label combinations before accepting a new series.
 - Decision: admission control on new series is a gateway function with a per-tenant budget, because index memory is shared and cannot be reclaimed quickly.

Architecture and flow — IC3 baseline



- **Agent**
 - Buys concurrent-safe batching through an atomic buffer swap and a bounded spool.
 - Cost: bounded local loss before acceptance under the documented drop policy.
 - **Gateway**
 - Validates schema and payload bounds before durable acceptance.
 - Applies tenant admission budgets.
 - Acknowledges only after the log reports the required durability condition.
 - Buys a single admission point for cardinality and tenancy.
 - Cost: one extra hop and the 250 ms acknowledgment budget must include the log quorum write.
 - **Durable log**
 - Decouples short storage outages from ingestion.
 - Supplies ordered partitions and replay.
 - It has finite retention and capacity.
 - Cost: retention bytes, partition management, and a second system to operate.
 - **Consumers**
 - Raw-point and rollup consumers apply idempotent writes and then advance recoverable progress.
 - Buys replay safety and independent scaling of raw versus rollup paths.
 - Cost: at-least-once delivery, so every sink write must carry a unique application identity.
 - **Query tier**
 - Uses independent workers and query budgets.
 - Separate processes do not automatically isolate shared disk, memory, or storage bandwidth.
 - Reserve storage capacity or use isolated read resources for expensive queries.
 - Cost: bounded results and freshness metadata replace the illusion of complete, current data.
 - **Control plane versus data plane**
 - Series catalog, tenant budgets, and retention policies live in the control plane.
 - Gateways and consumers hold cached snapshots so acceptance continues when the catalog is unavailable.
 - Cost: a budget change propagates with snapshot delay rather than instantly.
- Atomically swap the agent's active buffer for a fresh buffer.
- Serialize the detached batch outside its short lock.

- Send a stable batch identity with retry and jitter.
- Validate and durably append at the gateway.
- Acknowledge acceptance and release the local retry copy.
- Consumers apply idempotent writes before advancing recoverable progress.
- Queries read committed raw points or compatible rollups with freshness metadata.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
POST /v1/metric-batches	Batch ID, producer incarnation, sequence, points	202 after durable append; same batch ID returns the prior result; partial acceptance is reported explicitly or the whole batch fails
GET /v1/metric-batches/{producer}/{batch}	Acceptance and processing outcome within retention	404 after retention; authoritative answer after a lost 202
GET /v1/query	Tenant-scoped selector, range, resolution	422 over fan-out or point caps; response carries freshness watermark
GET /v1/series	Paginated names/labels	Prevent arbitrary cross-tenant enumeration; cursor scoped to tenant
POST /v1/replays	Restricted source range and transform version	Idempotency key, rate limit, and audit; 409 on overlapping active replay

State	Identity/key	Invariant
Series	(tenant, canonical name, sorted labels)	Full canonical identity resolves hash collisions
Logical point	(series, producer incarnation, sample sequence)	Retry keeps identity; source timestamp alone may collide
Log record	(partition, offset)	Recovery coordinate for this append, not cross-replay business identity
Rollup contribution	(window, series, source batch ID)	Unique application boundary prevents double addition
Materialized window	(tenant, series, window start, resolution, version)	Rebuild can replace a complete version atomically

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Logical points and log records live in the partitioned log and the TSDB; SQL holds the series catalog, batch outcomes, rollup contributions and windows, and replay records.

```

CREATE TABLE metric_series (
  tenant_id text NOT NULL, series_id text NOT NULL,          -- hash of the canonical identity; never
  trusted alone
  name text NOT NULL, canonical_labels text NOT NULL,        -- sorted label pairs; resolves hash
  collisions
  labels jsonb NOT NULL, created_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, series_id),
  UNIQUE (tenant_id, name, canonical_labels)
);
CREATE INDEX metric_series_by_name ON metric_series (tenant_id, name, series_id);
CREATE TABLE metric_batches (
  tenant_id text NOT NULL, producer_id text NOT NULL, batch_id text NOT NULL,
  producer_incarnation text NOT NULL, sequence bigint NOT NULL CHECK (sequence >= 0),
  payload_hash text NOT NULL,                                -- same batch_id with a different hash is
  409
  state text NOT NULL CHECK (state IN ('ACCEPTED','PARTIAL','REJECTED','PROCESSED')),
  accepted_count integer NOT NULL CHECK (accepted_count >= 0),
  rejected_count integer NOT NULL CHECK (rejected_count >= 0),
  accepted_at timestamptz NOT NULL, expires_at timestamptz NOT NULL, -- 404 after retention
  PRIMARY KEY (tenant_id, producer_id, batch_id),            -- lost 202: the retry reads the prior
  result
  CHECK (expires_at > accepted_at)
);
CREATE INDEX metric_batches_expiry ON metric_batches (expires_at);
CREATE TABLE rollup_contributions (
  tenant_id text NOT NULL, series_id text NOT NULL,
  resolution_s integer NOT NULL CHECK (resolution_s > 0), window_start timestamptz NOT NULL,
  source_batch_id text NOT NULL,                             -- unique application boundary
  sample_count bigint NOT NULL CHECK (sample_count >= 0),
  sum_value double precision NOT NULL, min_value double precision NOT NULL, max_value double precision NOT
  NULL,
  PRIMARY KEY (tenant_id, series_id, resolution_s, window_start, source_batch_id), -- duplicate insert is a
  no-op
  FOREIGN KEY (tenant_id, series_id) REFERENCES metric_series
);
CREATE TABLE rollup_windows (
  tenant_id text NOT NULL, series_id text NOT NULL,
  resolution_s integer NOT NULL CHECK (resolution_s > 0), window_start timestamptz NOT NULL,
  version bigint NOT NULL CHECK (version > 0),                -- a rebuild replaces a complete version
  atomically
  sample_count bigint NOT NULL CHECK (sample_count >= 0),
  sum_value double precision NOT NULL, min_value double precision NOT NULL, max_value double precision NOT
  NULL,
  histogram jsonb,                                           -- mergeable bucket counts, never
  percentiles
  provisional boolean NOT NULL,                               -- true until the lateness window closes
  PRIMARY KEY (tenant_id, series_id, resolution_s, window_start, version),
  FOREIGN KEY (tenant_id, series_id) REFERENCES metric_series
);
CREATE TABLE metric_replays (
  tenant_id text NOT NULL, replay_id text NOT NULL, idempotency_key text NOT NULL,
  source_from timestamptz NOT NULL, source_to timestamptz NOT NULL,
  transform_version text NOT NULL, actor_id text NOT NULL,
  state text NOT NULL CHECK (state IN ('ACCEPTED','RUNNING','SUCCEEDED','FAILED','CANCELLED')),
  created_at timestamptz NOT NULL, updated_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, replay_id),
  UNIQUE (tenant_id, idempotency_key),
  CHECK (source_to > source_from)
);
CREATE INDEX metric_replays_active ON metric_replays (tenant_id, source_from, source_to)
  WHERE state IN ('ACCEPTED','RUNNING');                      -- overlap check behind the 409

```

- Not captured by the DDL
 - Logical points (series, producer incarnation, sample sequence) are TSDB samples partitioned by (tenant, series_id, time_bucket); log records (partition, offset) are recovery coordinates that never become rows.

- Consumer progress is the committed log offset; `rollup_contributions` is the correctness boundary that absorbs a redelivered record.
- Tenant series and label budgets are control-plane configuration that gateways cache as snapshots; enforcing them is a count against `metric_series`, not a constraint.
- Deviation from the companion
 - The file's series identity (tenant, canonical name, sorted labels) is the `UNIQUE` constraint; `series_id` stays the primary key so the rollup tables carry a compact foreign key.
 - `metric_batches` gains `producer_incarnation`, `sequence`, `counts`, and `expires_at` for the 202 and 404-after-retention contract; `rollup_contributions`, `rollup_windows`, and `metric_replays` are added for the state table rows and `POST /replays`.
 - The replay overlap check is a partial index plus an application query; a range `EXCLUDE` constraint is the stricter alternative.
- Partition by stable series identity.
 - A consumer may own multiple partitions.
 - Ordinary consumer-group parallelism is bounded by partition count.
 - Splitting one series across partitions abandons simple per-series ordering.
- Batch routing
 - A host batch may contain many series.
 - Gateway can fan records into their series partitions.
 - Partial append failure requires retry-safe point identity.
 - Do not acknowledge a whole batch when only some required records are durable unless the API reports partial acceptance.
 - OCI Monitoring's `PostMetricData` exposes a batch-atomicity choice for this exact case
 - Treat that as documented OCI behavior and the routing above as proposed design.

Agent concurrency and aggregation — IC3 baseline

```

RECORD(point):
    under short buffer lock:
        validate bounded series count
        apply type-specific aggregation to active buffer

FLUSH():
    under same lock:
        detached = active buffer
        active buffer = new empty buffer
    serialize detached outside lock
    persist/send with stable batch ID
    release retry copy only after durable acceptance
  
```

- Never copy a concurrent map and then remove copied keys as a flush protocol.
 - Writers can update those keys between copy and remove.
 - The removal can discard points that were never included in the sent batch.
- Bound spool bytes and batch age.
 - A row-count limit does not bound memory when labels and payloads vary.
 - Concrete assumption: 64 MiB spool per host and 5 min maximum batch age.
 - Drop low-priority unaccepted telemetry with explicit counters when disk is exhausted.
 - Critical audit or business records require a different contract.
- Type-specific aggregation
 - Counter deltas retain producer reset/incarnation information.
 - Gauges use a declared last-value or min/max sampling policy.
 - Histogram buckets merge only when boundaries are compatible.
 - Average equals total sum divided by total count.

- Averaging independent percentiles does not yield the combined percentile.
- Cardinality
 - Normalize route templates.
 - Reject unbounded user IDs or request IDs as labels.
 - Apply a per-tenant active-series budget.
 - See [Prometheus instrumentation guidance](#).
- Exact failure
 - The lock protects the swap, not the send.
 - A crash after the swap but before durable spooling can lose the detached batch under the stated metrics policy.
 - A dead process cannot reliably increment its own drop counter afterward.
 - Infer the gap from producer sequences or an independent checkpoint, or disclose that exact loss counting is unavailable.

Delivery, time, and replay — IC3 baseline

- Producer retries
 - An idempotent broker producer reduces duplicates within its supported scope.
 - Gateway retries and explicit later replay still need stable application identities.
- Sink transaction
 - Insert a unique contribution and update its aggregate in one transaction when supported.
 - On duplicate contribution, do not add again.
 - Commit consumer progress only after the sink state is recoverable.
 - Alternatively materialize deterministic complete windows and replace by version.
- Consistency boundary
 - The log offset is the recovery coordinate
 - The sink's unique contribution key is the correctness boundary.
 - A consumer that commits the offset first and writes second can lose data
 - A consumer that writes first and commits second can duplicate, which the unique key absorbs.
- Event time
 - Keep source observation time and gateway receipt time separate.
 - Accept bounded out-of-order points, for example within 5 min by assumption.
 - Expose provisional rollups until the lateness window closes.
 - Late corrections create a new materialization version or enter a repair workflow.
- Poison versus outage
 - Deterministic schema errors enter quarantine with diagnostic metadata.
 - Sink unavailability should pause/retry with backpressure.
 - Moving every record to DLQ during an outage merely relocates the backlog.
- Kafka transactions do not automatically make an external TSDB exactly once.
 - The external sink must cooperate through idempotency or an atomic progress boundary.
 - See [Kafka delivery semantics](#).

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Agent disconnected	One host	Spool age and retry rate	Bounded local buffering; counted drops when policy permits	Retry stable batches with jitter
Gateway fault domain lost	One fault domain's share of requests	Health checks, error rate	Remaining domains absorb load; agents retry	Load balancer drains; replace capacity

Failure	Blast radius	Detection	Degraded behavior	Recovery
Log quorum unavailable	A partition cohort	Produce failures	Stop durable acknowledgment	Recover replicas and replay local spool
TSDB unavailable	Its assigned series	Consumer lag age	Pause consumption; query freshness degrades	Catch up below resource limits
Series catalog / policy store unavailable (control plane)	New budgets and new-series admission	Snapshot age	Gateways accept on cached budgets; unknown new series held to the cached quota	Refresh snapshots; reconcile series created during the outage
High-cardinality tenant	Shared index resources	New-series admission and memory	Reject new series over budget	Correct instrumentation and expire unused series
Poison record	A bounded record set	Classified validation failures	Quarantine without infinite retry	Replay with stable original identity
Duplicate replay	Target series/windows	Unique-key conflicts	No double aggregate	Audit replay progress and reconcile counts
Expensive query	Query/storage resources	Query cost and queue age	Cancel or return bounded partial results by contract	Tighten fan-out and resolution limits

- Observe
 - Oldest unprocessed record age.
 - Estimated time to retention loss.
 - End-to-end freshness.
 - Spool drops.
 - Quarantine growth.
 - Tenant cardinality admission.
- Retry discipline
 - Agents retry with exponential backoff and jitter under a total elapsed deadline.
 - Retry only transport and 5xx responses
 - A 4xx validation error requires a fix, not a retry.
 - Consumers retry sink writes with backpressure
 - They never skip an offset to make progress.
- Tenant isolation
 - Authenticate producers and bind tenant identity server-side.
 - Apply authorization to labels, queries, and replay operations.
 - Per-tenant budgets cover ingest rate, active series, and query cost.
 - A tenant's query receives 404 on another tenant's series, not 403, to avoid existence leaks.
 - Remove credentials and sensitive payloads from diagnostic logs.
 - Encrypt transport and durable stores.
- Rollout and rollback safety
 - Roll the gateway fleet by fault domain
 - Agents keep spooling, so a bad deploy costs freshness, not accepted data.
 - Consumer schema changes follow expand, migrate, contract
 - A new consumer must read records written by the old gateway.
 - Rollback of a rollout transform is a new materialization version, never an in-place edit.
- Operational drill
 - Stop a sink for a bounded interval.
 - Measure backlog growth and net catch-up rate.
 - Verify replay does not double aggregates.
 - Confirm queries expose stale data instead of silently implying freshness.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Direct TSDB write	Synchronous ingestion	Small topology	Sink outages directly affect producers	Low volume and durable producer buffering
Partitioned replay log	Durable append before sinks	Replay and independent consumers	Retention, partitions, and operating cost	Baseline sustained shared ingestion
Work queue	Task delivery	Simple worker decoupling	Replay and ordering depend on queue contract	One task consumer without historical reprocessing
Raw-only storage	Preserve all detail	Flexible future queries	Storage and read amplification	Small retention or forensic requirements
Raw plus rollups	Multiple resolutions	Lower query/storage cost	Late correction and versioning complexity	Repeated time-range dashboards

- IC4 stretch — no accepted loss
 - Rank accepted-data recoverability before query freshness.
 - Remove drop-on-full for accepted records
 - The agent blocks or returns backpressure instead.
 - Reduce raw retention or query precision only with an explicit product agreement.
 - Separate billable metering from lossy operational telemetry.
- IC4 stretch — 10× load
 - Raw daily input becomes approximately 43.2 TB/day before replication.
 - Local aggregation and retention choices can save more than adding brokers.
 - Preserve metric semantics while reducing resolution.
- IC4 stretch — cost and blast radius
 - Isolate large tenants or workload classes into cells.
 - Bound log retention pressure and query blast radius per cell.
 - Migrate series ownership with a routing version and a replay boundary.
 - Replicated log retention is the dominant storage cost
 - Shorten it only after the catch-up drill proves the margin.
- Weakest assumption
 - The sink absorbs 1.5× live rate during catch-up while still serving queries.
 - If the real headroom is 1.1×, a 2 h outage takes 20 h to drain and consumes nearly the whole 24 h retention window.
 - Mitigation: measure catch-up under query load, shed rollup work during catch-up, and alarm on estimated time to retention loss.
- Over-engineering warning
 - Do not call the pipeline “zero loss” without naming the acknowledgment boundary and surviving failure set.
 - Do not add Kafka transactions expecting them to make the external database exactly once.
- Related business prompt
 - [Healthcare ingestion](#) has stricter record retention and reconciliation requirements.
 - Its accepted business records must not inherit the metrics drop policy.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Metric types, loss policy, freshness target, tenant in every key	Separates billing-grade from telemetry and names what the drop policy buys	Negotiates the loss contract instead of assuming it
Estimates	One million points/s, backlog and catch-up math, cardinality	Lets catch-up headroom set retention; names the correlated-outage case	The catch-up number changes the retention decision

Phase	IC3 must land	IC4 adds	Separating signal
Architecture	Gateway, log, consumers, query tier with one trace	Control-plane snapshots so ingestion survives a catalog outage	Every component has a mechanism and a price
Deep dive	Swap-buffer flush, stable point identity, unique contribution key	Offset-first versus write-first consumer failure analysis; versioned rollups	Reasons across the log and sink boundary
Operations	Failure table, freshness alarms, outage drill	Cells, fault-domain rollout, retention as a cost control	Volunteers the drill before being asked
Wrap	Recap and tradeoff	Names the catch-up assumption as the weakest first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Does batching reduce one million points to ten thousand points?
 - **IC3 baseline**
 - No
 - This example reduces HTTP requests to ten thousand per second.
 - **IC4 stretch**
 - Only semantically valid aggregation can reduce logical observations.
- Q2: A consumer crashes after writing but before committing its offset.
 - **IC3 baseline**
 - It reprocesses
 - A unique sink identity prevents double application.
 - **IC4 stretch**
 - An aggregate needs atomic dedup-plus-update, not a separate best-effort dedup cache.
- Q3: Can we average host p99 latency values?
 - **IC3 baseline**
 - No
 - Merge distributions or compatible histogram buckets.
 - **IC4 stretch**
 - Choose resolution and error budget before comparing histogram formats.
- Q4: The data is now billing-grade with no accepted loss.
 - **IC3 baseline**
 - Remove drop-on-full for accepted records and enforce durable acceptance/backpressure.
 - **IC4 stretch**
 - Design reconciliation and regional durability separately from dashboard telemetry.
- Q5: Is one partition per consumer mandatory?
 - **IC3 baseline**
 - A consumer can own several partitions
 - One partition has one active group owner.
 - **IC4 stretch**
 - Partition reassignment must coordinate progress and sink effects.
- Q6: Why is query deployment isolation insufficient?
 - **IC3 baseline**
 - Ingestion and queries may still compete for the same storage resources.
 - **IC4 stretch**
 - Reserve resources or isolate read replicas/materialized blocks.
- Q7 (premise reversal): Why not drop the log and write directly to the TSDB with a bigger agent spool?
 - **IC3 baseline**
 - A sink outage then becomes 100,000 independent recoveries, replay has no shared ordered source, and the agent spool is the weakest disk in the fleet.

- **IC4 stretch**
 - Acceptable only for low volume with a durable producer store
 - At one million points/s the log's cost is cheaper than fleet-wide spool operations.
- Q8: The series catalog is down
 - Do we stop accepting points?
 - **IC3 baseline**
 - No
 - Gateways accept on cached budgets and reconcile new series after the catalog returns.
 - **IC4 stretch**
 - Bound the reconciliation debt with a cached quota for unknown series so the outage cannot become a cardinality bypass.
- Validation scenarios
 - Agent record racing with flush.
 - Partial batch append and retry.
 - Sink write followed by consumer crash.
 - Replay of an existing aggregate contribution.
 - Producer counter reset.
 - Late samples after a rollup closes.
 - Recovery before log retention expires.
 - Catalog outage while a tenant creates new series.

Recall card

- "The agent batches without losing concurrent writes."
- "Durable append separates acceptance from query freshness."
- "Stable point identity and an atomic sink boundary make replay safe."
- "Cardinality, retention, and catch-up capacity are first-class budgets."
- "The catalog is control plane; acceptance must not depend on it."
- Wrap-up script
 - "The design is a swap-buffer agent, a gateway that acknowledges after a durable log quorum, and idempotent consumers feeding a TSDB with rollups."
 - "An acknowledgment marks durable log acceptance, and sink-side contribution identity protects replay only when stored atomically with aggregation."
 - "The main tradeoff is the log itself: it buys replay and outage isolation at the price of retention bytes and a second system to run."
 - "My weakest assumption is 1.5× catch-up headroom; I would run the sink-outage drill first to validate it."

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Kafka design](#): partitioned delivery and the external-sink boundary.
- [Prometheus instrumentation](#): cardinality and metric semantics.
- [Prometheus metric types](#): counters, gauges, and distribution metrics.
- [OCI Monitoring PostMetricData](#): batch atomicity option for partial acceptance
 - Documented OCI behavior, not this design.